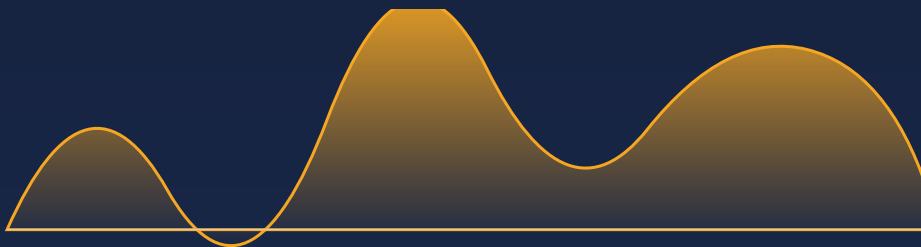


Solar Energy Prediction and Analysis Platform

A location-aware solar generation calculator for consumers, and a rigorous hourly-irradiance analysis console for researchers — sharing one physical engine, one data source, and one philosophy: every number traces back to an assumption, and every assumption is on the table.



Codename: Helios · **Version:** 1.0 · **Document type:** Technical project documentation

Repository root: college_project/ · **Backend:** Python 3.11 + FastAPI · **Frontend:** Next.js 15 + React 19

Compiled: 2026-08-21

Table of Contents

1	Executive Summary	4
2	Introduction and Background	6
3	Problem Statement	7
4	Objectives	8
5	Target Users and Use Cases	9
6	System Overview	11
7	System Architecture	12
8	Technology Stack	14
9	Repository Structure	16
10	User Experience and Workflow	18
11	Adaptive Questionnaire	19
12	User-Type Logic	20
13	Location System	21
14	Energy Consumption Model	23
15	Solar Panel System	24
16	Panel Quantity and System Capacity	26
17	Solar Energy Prediction Methodology	28
18	Mathematical Model — Formulas Used	30
19	Machine Learning Model (Analysis Console)	33
20	Weather and Solar Data	36
21	Data Flow	37
22	Results and Visualisation	38
23	Uncertainty and Limitations	39
24	Validation and Error Handling	41
25	Security	43
26	Performance and Responsiveness	44
27	Accessibility and Usability	45
28	Testing	46
29	Deployment	48
30	Installation and Running	50

31	Future Improvements	51
32	Conclusion	52
33	Glossary	53
34	References	55
35	Appendix — API and Configuration	56

Figures and tables are numbered per-section (e.g. Figure 7.1) and listed inline alongside the text they accompany.

1 Executive Summary

The **Solar Energy Prediction and Analysis Platform** (internal codename *Helios*) is a full-stack web application that helps a non-specialist decide whether to install a solar photovoltaic (PV) system at a specific location, and gives a researcher the tools to interrogate hourly irradiance forecasting on the same data. It exposes two surfaces over one shared backend:

- **A consumer calculator** (route `/`): a two-minute guided interview that returns expected annual generation, monthly and daily shapes, a confidence-rated range, an energy balance against the user's consumption, financial savings, payback, emissions avoided, and a shareable report — without asking the user to know what a kilowatt-peak is.
- **An analysis console** (route `/advanced`): a fourteen-view research workbench that trains scikit-learn models on ERA5 reanalysis, validates them with time-aware splits and an embargo gap, calibrates prediction intervals with Conformalised Quantile Regression, and persists every run as a reproducible experiment.

Both surfaces run against the same deterministic physical chain (Erbs → HDKR → Faiman → PVWatts v5) and the same weather source (Open-Meteo Historical Weather API / ERA5 reanalysis). Machine learning is deliberately restricted to the forward-looking research path where predicting a specific future day is the actual task; the consumer answer is a climatology question, computed by running years of observed weather through the physics.

Problem addressed

Solar generation depends on location, orientation, weather, panel technology and the user's own consumption pattern. Existing reference apps often return the same number for morning and afternoon inputs or state impossible pairs like "16.79 kWh at 0 % irradiance". The platform's remit is to return an answer that is location-aware, time-consistent, physically defensible, and honestly bounded.

Major capabilities

- Address search, GPS auto-locate, map-pin, and reverse geocoding.
- Three ways to describe consumption (monthly bill, monthly kWh, or itemised equipment) and three ways to describe available space (single value, length × width, or polygon on a map).
- Automatic tilt/azimuth search against the location's own weather.
- Panel-count-first sizing, honouring existing systems and comparing scenarios.
- Regional tariffs and currencies for 30+ countries; farm-subsidy detection.
- Hourly self-consumption, battery dispatch and sizing, grid import/export ledger.
- Full economics: CAPEX, savings, payback, ROI, LCOE, degradation-corrected lifetime yield, CO₂ avoided with plain-language equivalences.
- Measured uncertainty bands built from inter-annual weather variability and named assumption components; monthly ranges reflect each month's own spread.
- Analysis console: 10 model families, rolling-origin cross-validation, five-check leakage audit, permutation importance, partial dependence, weather-regime error decomposition, anomaly detection, scenario analysis, and a Markdown report generator.

Outcome

A single-repository application (`backend/` + `frontend/`) that runs locally with no API keys, produces a saved estimate in about a second against cached weather (up to ~15 s for a location never seen before), and completes a full model-trained analysis in 10–40 s. Backed by 223+ backend and 51 frontend automated tests.

2 Introduction and Background

Photovoltaic energy is the fastest-growing electricity source globally, but its output is weather-dependent and therefore uncertain. That uncertainty makes three practical decisions difficult: *sizing* a system to match consumption, *scheduling* maintenance around expected generation, and *balancing* a grid that increasingly relies on variable resources. A prediction of solar generation is only useful if it is genuinely predictive of the future, states how uncertain it is, and has been validated in a way that mirrors deployment conditions.

Existing consumer solar tools mostly use a "sunlight × capacity" rule of thumb: they take a monthly kWh/m² figure, multiply by system size, and print a number. That approach hides four problems: (i) it treats a national average as if it applied to a specific rooftop, (ii) it does not respond to time or orientation, (iii) it presents modelled figures without any range around them, and (iv) it cannot explain where its numbers came from. This platform was built to replace that class of tool with something that is transparent, location-aware and honest about what it does not know.

Why prediction is useful

- **Sizing.** The right system size is the smallest one that meets the user's realistic offset target — not the biggest one that fits the roof. Getting this wrong wastes capital or under-produces for the life of the system.
- **Consumption matching.** Solar generates when the sun is up. A household whose load is dominated by evening cooking recovers only a fraction of unstored generation through self-consumption; a farm whose pumps run during the day recovers most of it.
- **Financial planning.** Payback depends on tariff, degradation, export policy and inter-annual weather variability; each of these is genuinely variable and each is stated openly here.
- **Grid operations.** For an operator, hour-by-hour forecast accuracy with a calibrated range is the difference between a reserve that meets contingency and one that leaves a gap.

Challenges in estimating solar generation

- **Atmospheric attenuation is stochastic.** Cloud, aerosol, humidity and precipitation move irradiance around by up to an order of magnitude within a single day.
- **Solar geometry is deterministic.** Sun position, angle of incidence and clear-sky reference are known analytically; mixing them into a learning problem lets the learner memorise the clock instead of the atmosphere.
- **Reanalysis is not measurement.** Grid-cell averages differ systematically from point observations; this must be surfaced, not hidden.
- **Consumption archetypes are approximations.** Real interval-meter data would beat any archetype, and the platform says so when it uses one.

3 Problem Statement

Anyone considering solar — a farmer, a homeowner, a shopkeeper, a facility manager — needs to answer four practical questions before spending money:

1. How much energy can I generate here, per day and per year?
2. How much of my consumption does that offset, and when does the mismatch fall?
3. What will it cost, and how long will it take to pay back?
4. How confident am I in each of these answers, given what I do and do not know?

The available tools fail these questions in three characteristic ways. First, they return one national or regional number rather than one specific to the user's coordinates — so a farm in a monsoon belt and a farm in a desert receive the same result. Second, they report a single point estimate as if it were measured, giving no signal that the actual value in any particular year may be 15 % above or below it. Third, they show none of the assumptions behind the number, so the user has no way to sanity-check it or to update it as their situation changes.

What the platform must therefore do

- **Answer per-location.** Every calculation must run against the actual weather record at the supplied coordinates, not a regional lookup.
- **Answer per-user.** The interview must adapt to what the user is trying to do — a farm question is not a home question — and must let every technical question be declined without producing a garbage answer.
- **Attach a range.** Every headline number must carry an honest uncertainty band built from named components, not a round $\pm 10\%$.
- **Show its working.** Every assumption used must be recorded and displayed, and changing any of them must move every dependent number.
- **Refuse to fabricate.** Where the platform cannot know something (e.g. because no metered generation was supplied to validate against), it must say so in the interface rather than silently substituting a default.

Reference-app failure that shaped this brief: The predecessor application returned *16.79 kWh alongside 0 % stated irradiance* and gave the *same figure for 06:51 and 13:51*. Both are recorded in `docs/REFERENCE_AUDIT.md`. The platform's design decisions — time-aware validation, physical constraints applied after inference, explicit assumption ledger — exist so that neither failure can recur in Helios.

4 Objectives

Primary objectives

1. **Deliver location-specific solar generation estimates** using the observed hourly weather record at the user's supplied coordinates, not a regional average.
2. **Adapt the questionnaire** to six user categories (home, farm, shop, commercial/industrial, institution, exploring) with different questions, defaults and result emphasis.
3. **Attach measured uncertainty** to every headline number, decomposed into weather variability, resource error, model error and per-assumption components.
4. **Compute an energy balance and finances**: hourly self-consumption and export, battery dispatch/sizing where requested, tariff-aware savings, payback, ROI, LCOE and lifetime CO₂ avoided.
5. **Support three goals**: sizing a planned system, describing an existing one, and comparing scenarios side-by-side.
6. **Produce a shareable, saved estimate** with a stable URL, a plain-text report and an editable assumption ledger.
7. **Provide a research console** that trains and evaluates ten scikit-learn model families with time-aware validation, prediction intervals and reproducibility manifests.

Secondary objectives

1. **Run keyless**. Every upstream service used (Open-Meteo Archive and Geocoding, OpenStreetMap Nominatim) is public; no API keys and no accounts are required.
2. **Localise gracefully**. Interview strings are held as data (not in components) so a future translator can add Telugu or Tamil without editing React.
3. **Fit into a rural context**. Mobile-first layout, touch-friendly controls, tolerance of dropped GPS permission, an offline-tolerant map with a Leaflet swap.
4. **Never present modelled figures as measurements**. No metered PV output was available, so the interface labels PV figures as physically modelled and uses no synthetic "validation" against fabricated data.
5. **Keep the surfaces separate**. The consumer flow and the console flow do not share a palette, a route, a wording register, or a reader.
6. **Preserve provenance**. Every weather fetch is cached to disk and every model run is written to an experiment store with a dataset fingerprint (SHA-256 of the feature matrix + target + names + period).

5 Target Users and Use Cases

Six user categories are supported explicitly by the interview data in

`backend/app/estimate/personas.py`. Each carries its own icon, its own default question set, and its own emphasis in the results.

User type	Typical inputs collected	Result emphasis
Home	Monthly bill or kWh, roof space or dimensions, occupants (optional), appliance list (optional), tariff (regional default, editable).	Bill offset, monthly savings, payback, indicative panel count, battery option if selected.
Farm / Agriculture	What is being powered (pumps, cold storage, sheds, dairy, poultry), pump horsepower and head, required daily water volume, crop water requirement (mm/day), land or shed area.	Daily pumping hours (best month, worst month, annual mean), water pumped per day (m ³), irrigable area (ha), subsidy-aware savings note.
Shop / Small Business	Business type (shop / clinic / salon / restaurant / office), operating hours, monthly bill or kWh, available roof area, grid connection type.	Daytime self-consumption, offset percentage, payback under regional commercial tariff.
Commercial / Industrial	Facility type (factory / warehouse / cold storage / logistics / mill), peak demand and connected load (optional), operating hours, offset target percentage, grid connection.	Solar offset percentage, peak-demand caveat (solar rarely reduces peak-demand charges), self-consumption ratio, lifetime CO ₂ .
School / Institution	Facility type, weekday occupancy pattern (implicit through operating hours), roof area, monthly bill.	Term-time vs holiday balance, offset percentage, community-oriented CO ₂ equivalences (cars off the road, trees planted).
Just Exploring	Minimum: location only. Any consumption input is optional.	Site potential (specific yield, capacity factor, best/worst month), no financial figures unless a tariff and demand are supplied.

Table 5.1 — Six user types supported by the interview. Every user type flows through the same computational engine; the difference is which questions are asked, which defaults are applied, and which sections of the result are emphasised.

Representative workflows

Farmer with a 5 HP pump and 3 acres

Answer *Farm*, drop a pin on the well, tell the interview it is a 5 HP pump on a 20 m head. Everything else can be skipped. The estimate returns the daily generation, the number of pump-hours that generation supports, and how many m³ of water that moves in the driest month versus the monsoon. A subsidy-aware caveat appears if the location's default agricultural tariff is subsidised.

Homeowner with a 3 000 INR monthly bill and 40 m² of usable roof

Answer *Home*, enter the bill, enter the roof area. The engine picks the best tilt/azimuth for the location, recommends a panel count that hits the bill (subject to space), returns monthly savings and payback, and asks whether to add battery storage.

Facility manager comparing 100 kWp, 150 kWp, and 200 kWp options

Fill in the details once (facility type, roof, tariff), then use `POST /api/estimate/compare` to score all three capacities against the same weather, producing a table of annual kWh, offset %, CAPEX, payback and ROI.

Owner of an existing 5 kWp array

Set goal to *Existing*, enter the panel count and their rated wattage. The sizer is bypassed — capacity is a measurement of what is on the roof, not a recommendation — and the engine produces the true expected production of the installed system, giving the owner a benchmark against their actual bills.

6 System Overview

Helios is a two-tier web application. A Python 3.11 backend built on FastAPI exposes a typed JSON API; a Next.js 15 frontend consumes it. All state is persisted server-side as JSON on local disk — there is no database and no user account system. Two external services are called: the Open-Meteo Historical Weather API for hourly reanalysis, and OpenStreetMap Nominatim for reverse-geocoding a dropped map pin. Both are keyless.

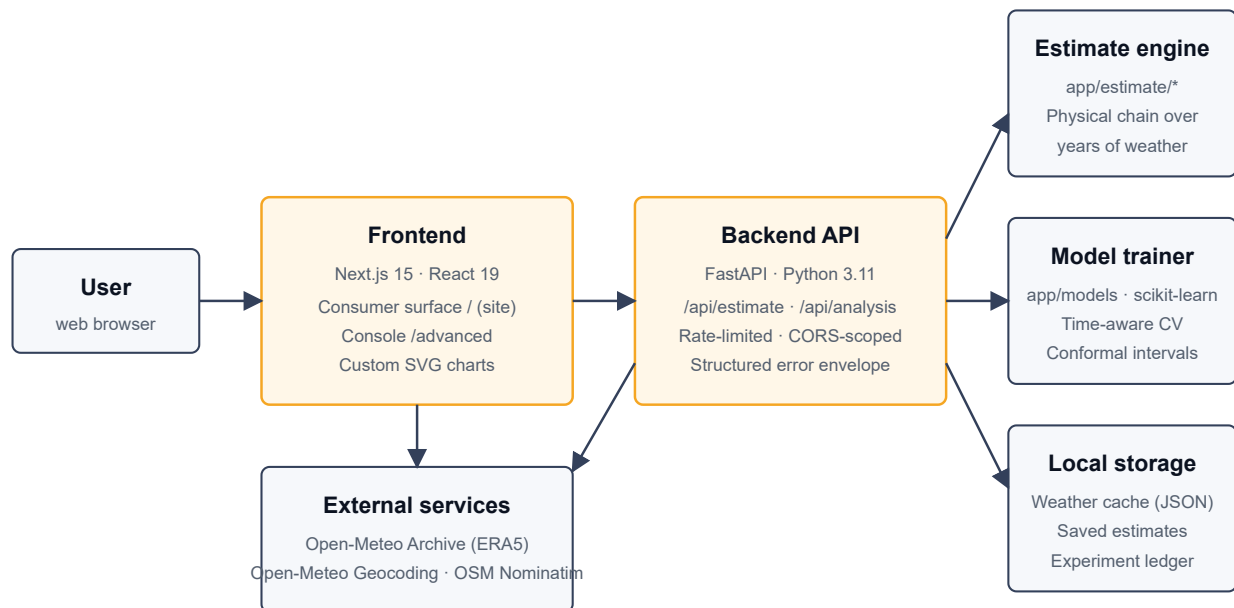


Figure 6.1 — High-level system view. The frontend never contacts external services directly — all upstream calls flow through the backend, which caches every response.

End-to-end flow (consumer estimate)

```

User → UI wizard → typed request → FastAPI /api/estimate
  → engine.run(input):
    resolve_location → tariff/currency → resolve_demand
    → fetch_history (weather cache) → optimal_orientation
    → reference climatology (per kWp) → recommend_capacity
    → panel-quantised system → final climatology
    → energy balance → economics
    → uncertainty → farm capability (if farm)
  → assembled payload → optionally save → response
  → React ResultView / CompareView / SystemSpecification
  
```

7 System Architecture

Presentation Layer

Next.js 15 App Router with React 19 and TypeScript. Two route groups keep the surfaces separate: `app/(site)/` hosts the consumer flow (landing, start, estimate, result, compare, resources, projects, how-it-works, about) with a light palette, while `app/advanced/` hosts the analysis console with a dark instrument palette and no shared chrome. Styling is Tailwind CSS 3.4 with a hand-tuned token scheme; there is no charting library — every plot in both surfaces is a hand-rolled SVG in `components/charts/` and `components/result/charts.tsx`.

Application / API Layer

FastAPI application `backend/app/main.py` assembles four routers:

- `app.api.routes.meta` — health, catalogue, interview metadata, data-source manifest, place search.
- `app.api.routes.estimate` — consumer surface: create/read/update/delete/rename an estimate, run a comparison, produce a professional report.
- `app.api.routes.analysis` — console surface: create an analysis, then query its quality, predictions, performance, intervals, explanation, anomalies, forecast, scenarios and report subresources.
- `app.api.routes.experiments` — list, compare and delete persisted experiments.

Cross-cutting concerns live in `main.py`: a fixed-window in-process rate limiter, a middleware that stamps every response with a request ID and elapsed time, security headers (`X-Content-Type-Options` , `Referrer-Policy` , `X-Frame-Options`), and error translators that turn Pydantic validation failures, upstream data-source errors and internal exceptions into a structured envelope with `error` , `message` , `detail` , `remedy` and (for validation) `field_errors` .

Data Layer

No relational database. Two on-disk stores keep server state:

- **Weather & geocoding cache** — JSON files keyed by a SHA-256 of the request URL + parameters, with a configurable TTL. Every upstream call goes through this cache, so an orientation search that evaluates twenty candidate tilts still fetches the weather only once.
- **Estimate store** — JSON records under `backend/var/estimates/` , addressable by opaque estimate ID. Records include the full payload, the original request snapshot (for update), the label and timestamps. A retention sweep at startup prunes records past the configurable window.
- **Experiment store** — JSON records for every training run, carrying the dataset fingerprint, metrics, folds, baselines and manifest; used by the console's *Experiments* view.

Prediction Layer

Two paths share a single physical chain and a single feature pipeline:

- **Fast path (consumer)** — `app/estimate/climatology.py` runs the physical chain over 3 years of ERA5 hourly weather at the coordinates. No model is trained; inter-annual variability is *measured*, not assumed.

- **Forward-looking path (console)** — `app/models/trainer.py` trains a scikit-learn estimator on a feature set built from the same weather (see § 19), splits chronologically with a 24-hour embargo, cross-validates with rolling origins, and calibrates a prediction interval using Conformalised Quantile Regression.

External Services

Service	Endpoint	Purpose	Key required
Open-Meteo Historical Weather	<code>archive-api.open-meteo.com/v1/archive</code>	Hourly ERA5 / ERA5-Land reanalysis: GHI, DNI, DHI, T, RH, wind, cloud, precipitation, pressure.	No
Open-Meteo Geocoding	<code>geocoding-api.open-meteo.com/v1/search</code>	Place-name search and elevation (GeoNames source, CC-BY 4.0).	No
OpenStreetMap Nominatim	<code>nominatim.openstreetmap.org/reverse</code>	Reverse-geocoding a coordinate to a place name (map pin or GPS).	No
Open-Meteo Forecast	<code>api.open-meteo.com/v1/forecast</code>	Short-horizon forecast used by the console's <i>Forecast</i> view.	No

Table 7.1 — External services. Every one is keyless; the rate limit is entirely the provider's.

Design principle: *The frontend has no secrets and does no data acquisition.* Every upstream call is proxied through the backend so that a change of provider touches one file (`backend/app/data/sources.py`) and so that cache and quality scoring apply uniformly.

8 Technology Stack

Layer	Technology	Purpose
Frontend framework	Next.js 15.5 (App Router) + React 19 + TypeScript 5.7	Consumer surface and analysis console; server components for the shell, client components for interactivity.
Styling	Tailwind CSS 3.4 with custom tokens; PostCSS + Autoprefixer	Design-token-driven styling; two palettes (light for consumer, dark for console).
Map (default)	Hand-rolled Web-Mercator tile map (~250 LOC) — <code>components/estimate/map/TileMap.tsx</code>	Zero-bundle map with pin-drop and polygon-draw; renders raster tiles.
Map (swap)	Leaflet 1.9 via a dynamically-imported adapter	One-line environment switch (<code>NEXT_PUBLIC_MAP_ENGINE=leaflet</code>) for devices the built-in map mishandles.
Client state	React state + <code>lib/draft.ts</code> localStorage draft persistence	No global state library; the wizard's answers survive a page refresh.
Testing (frontend)	Vitest 2.1 + Testing Library + jsdom	51 tests covering projection maths, draft mapping and component promises.
Backend framework	FastAPI 0.115 + Uvicorn 0.32 (ASGI)	Typed JSON API with automatic OpenAPI docs at <code>/api/docs</code> .
Backend runtime	Python 3.11+	Type-hinted dataclasses across the domain layer.
Scientific stack	NumPy 1.26+, pandas 2.2+, SciPy 1.13+, scikit-learn 1.5+	Vectorised physical chain, feature engineering, model training and evaluation.
Data validation	Pydantic 2.9	Request-schema validation, error messages mapped to user-facing prompts.
Testing (backend)	pytest 8 + httpx 0.27	223+ tests including physical chain against astronomical values, split leakage, quality detection, API contracts.
Weather source	Open-Meteo Historical Weather API (ERA5 / ERA5-Land)	Hourly meteorological variables at the user's coordinates.
Geocoding	Open-Meteo Geocoding + OpenStreetMap Nominatim	Forward search (place → coords), reverse (coords → place).
Storage	Local filesystem (JSON)	Cache, saved estimates, experiment ledger. No database, no accounts.
Deployment	Uvicorn workers (backend) + Next.js production server (frontend)	Standard ASGI + Node processes; behind a reverse proxy in production.

Table 8.1 — Technology stack, as actually used in the repository. Nothing here is aspirational: each entry appears in `package.json` or `requirements.txt`.

Deliberate omissions

- **No charting library.** Every visualisation is a hand-written SVG so the design and print output are under full control and the bundle stays small.
- **No state-management library.** React state plus localStorage covers every wizard step.
- **No deep-learning runtime.** The model set intentionally stops at scikit-learn ensembles; sequence models (LSTM / BiGRU) are declared as future work in `app/models/registry.py::future_work()`.
- **No third-party auth.** There are no user accounts; estimates are addressable by an opaque, unlisted URL.

9 Repository Structure

```

college_project/
├── backend/
│   ├── app/
│   │   ├── main.py          FastAPI app, middleware, error translation
│   │   ├── config.py        Environment-driven settings (safe defaults)
│   │   └── api/
│   │       ├── service.py    Analysis creation + in-memory cache
│   │       └── routes/
│   │           ├── meta.py    /api/meta/* health, catalogue, interview
│   │           ├── estimate.py /api/estimate/* consumer surface (10 endpoints)
│   │           ├── analysis.py /api/analysis/* research surface (14 endpoints)
│   │           └── experiments.py /api/experiments/* persisted runs
│   ├── data/
│   │   └── sources.py        HTTP with disk cache; Open-Meteo & Nominatim clients
│   ├── features/
│   │   ├── solar_geometry.py NOAA position, Haurwitz, Erbs, HDKR, Faiman, PVWatts v5
│   │   ├── parameters.py     Solar-position features, clear-sky index, cyclical time
│   │   └── pipeline.py        Feature-set builder with leakage guards
│   ├── quality/engine.py     Transparent data-quality scoring (eight corruption types)
│   ├── models/
│   │   ├── registry.py       10 estimator specs with paper-cited hyperparameters
│   │   ├── trainer.py        Split, fit, evaluate, cross-validate, calibrate
│   │   ├── uncertainty.py     Conformalised quantile regression
│   │   └── forecast.py        Short-horizon forecasting for the console
│   ├── evaluation/
│   │   ├── metrics.py        MAE, RMSE, MBE, R2, rRMSE, pinball, PICP, PINAW, CRPS
│   │   ├── splitters.py       Chronological hold-out + rolling origins with embargo
│   │   ├── baselines.py       Persistence, smart persistence, climatology, seasonal-naive
│   │   └── leakage.py         Five-check audit (target, future, preprocessing, boundary,
│   │                           trivial)
│   ├── explain/attribution.py Permutation importance, partial dependence
│   ├── anomaly/detector.py    Physical, residual, ramp, drift detectors
│   ├── scenario/engine.py     Counterfactual scenarios with pathway decomposition
│   ├── experiments/store.py   On-disk experiment ledger
│   ├── reports/generator.py   Console-side Markdown report
│   ├── schemas/
│   │   └── estimate/          (see next tree)
│   ├── scripts/              calibrate_interval.py, evaluation utilities
│   └── tests/                 223+ pytest tests
├── frontend/
│   ├── app/
│   │   ├── layout.tsx        Root shell + font + globals
│   │   ├── globals.css       Tailwind base, tokens, print rules
│   │   └── (site)/
│   │       ├── page.tsx      Landing
│   │       ├── start/        User-type & goal picker
│   │       ├── estimate/     Interview wizard
│   │       ├── result/       Result dashboard
│   │       └── compare/       Scenario comparison

```

				projects/	Saved estimates gallery
				how-it-works/	Methodology page
				resources/	Solar resource explainer
				about/	About page
				advanced/	Analysis console (14 views)
				components/	
				Console.tsx	Console shell (dark palette)
				Setup.tsx	Console setup form
				site/	Nav, Footer, Icon, ProjectsDashboard, ResourceCurve
				estimate/	EstimateWizard, LocationStep, AreaField,
				EquipmentBuilder,	
					LoadingStages, StartFlow, fields, map/ (Tile + Leaflet)
				result/	ResultView, CompareView, PanelConfigurator,
					SystemSpecification, AssumptionEditor, charts
				charts/	Hand-rolled SVG chart primitives
				views/	14 console views (Overview, Performance, Uncertainty,
				...)	
				lib/	api client, draft, format, i18n, estimate mapping,
				useAsync	
				tests/	51 Vitest tests
				docs/	
				DELIVERY.md	Delivery report (what changed, methodology)
				RESEARCH_SYNTHESIS.md	Papers surveyed + design decisions
				PRODUCT_TRANSFORMATION_AUDIT.md	Consumer-layer audit & rationale
				REFERENCE_AUDIT.md	Failures of the predecessor app
				project_documentation/	This document (Markdown + HTML + PDF)

Estimate subpackage detail

backend/app/estimate/	
engine.py	1058 LOC · Pipeline orchestrator (14 numbered stages)
personas.py	1256 LOC · The interview as data (six user types, all questions)
sizing.py	667 LOC · Orientation search, capacity recommendation, panel
quantisation	
assumptions.py	641 LOC · Panel technologies, loss stack, all default values,
provenance	
demand.py	496 LOC · Bill / units / equipment / floor-area routes; hourly
archetypes	
climatology.py	344 LOC · Fast path: physical chain over N years of weather
balance.py	313 LOC · Hourly self-consumption, import/export, battery dispatch
uncertainty.py	319 LOC · Confidence band from named components
space.py	306 LOC · Panel packing, footprint, usable-fraction by mount type
tariffs.py	248 LOC · Regional currencies and default rates (30+ countries)
economics.py	248 LOC · CAPEX, savings, payback, ROI, LCOE, CO ₂
farm.py	237 LOC · Pumping hours, water pumped, irrigable area
store.py	230 LOC · JSON-file persistence for saved estimates
report.py	307 LOC · Plain-text professional report for hand-off

10 User Experience and Workflow

The consumer flow is a linear wizard with a single decision point at the top: what kind of user, and what goal. Every subsequent step adapts to those answers.

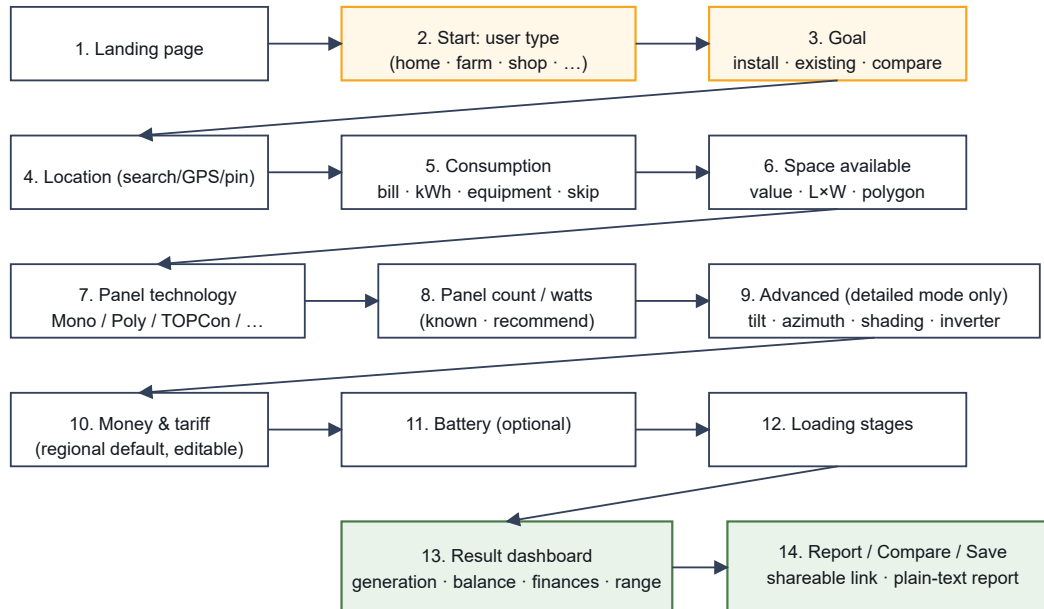


Figure 10.1 — End-to-end user workflow. The same result stage is reached by all six personas; the intermediate steps differ in question wording, defaults, and which cards are shown in the result.

Wizard behaviour

- **Skippable everywhere.** Every technical question includes an "I don't know" or "help me estimate" branch. Skipping is honest, not free: the assumption that fills the gap is recorded in the ledger, and the uncertainty band widens.
- **Draft persistence.** Answers are saved to `localStorage` as the user types (`lib/draft.ts`), so a page refresh, a phone lock or a network blip does not restart the interview.
- **Progressive disclosure.** Quick mode presents the essentials only; Detailed mode reveals orientation, inverter, loss overrides and degradation.
- **Deterministic loading stages.** The `LoadingStages` component reflects real work: location resolution → weather → orientation search → final yield → balance → economics. The stages are never marked finished before the backend reports them done (enforced by `frontend/tests/components.test.tsx`).

11 Adaptive Questionnaire

The interview is data, not code. `backend/app/estimate/personas.py` (1 256 lines) declares every question, its wording, its help text, a spoken form for a future voice interface, and — where the answer is technical — an explicit "I don't know" branch that names the assumption the platform will use instead. The React components render whatever `GET /api/meta/interview` returns.

Question kinds

Twelve question kinds cover the interview:

- `single_choice` radio-group selection
- `multi_choice` checkbox group
- `number` numeric input
- `currency` money input with regional symbol
- `area` area with unit switcher (m² / ft² / ac / ha)
- `location` place search + map + GPS
- `equipment_list` add-one-at-a-time appliance builder
- `pump_list` HP + head + hours entry
- `boolean` yes/no toggle
- `text` free text
- `info` read-only explainer card

Beginner mode vs Detailed mode

	Quick Estimate	Detailed Analysis
Consumption	Monthly bill or kWh only	Above plus equipment list / floor-area / pump list
Space	One number, one unit	Add length × width or polygon-on-map
Panel	Assumed technology, count derived	Choose technology, wattage, model, dimensions, V_{oc} , I_{sc} , V_{mp} , I_{mp}
Orientation	Auto-searched	Manual tilt / azimuth entry
Losses	PVWatts default stack	Per-loss overrides (dust, wiring, mismatch, availability, LID)
Inverter	DC/AC 1.2, η = 0.96	Explicit AC capacity or DC/AC ratio, η override
Lifetime & degradation	25 y, 0.5 %/y	Editable

Escape routes

Wherever the question is technical, at least one of these is offered:

- *I don't know* — *use the typical value*: engine substitutes the market-typical default, records it in the ledger with `provenance="market_typical"`, and widens the uncertainty band.
- *Help me estimate*: opens a sub-flow (e.g. equipment builder for consumption; L × W for area).
- *My installer said ...*: pass-through numeric with `provenance="user_supplied"`.

The transparency contract is enforced end-to-end: any figure appearing in the result has a corresponding row in the assumption ledger, and any ledger row with `editable=True` can be changed

post-hoc through `POST /api/estimate/{id}/update`, which re-runs the pipeline against the cached weather in about a second.

12 User-Type Logic

User type	Interview inputs actually collected	Result cards emphasised
home	location · monthly bill or kWh · roof area · panel tech (optional) · battery (optional) · shading level · tariff (regional)	Monthly savings · payback · offset % · CO ₂ avoided
farm	location · farm loads (multi) · pump HP + head · required daily water m ³ · crop water mm/day · monthly kWh (optional) · shed area (optional)	Daily pumping hours (best/worst month) · daily water m ³ · irrigable area · subsidy note
shop	location · business type · operating hours · bill or kWh · roof area · grid connection · tariff	Daytime self-consumption · offset % · payback · monthly savings
commercial	location · facility type · peak demand kW (optional) · connected load kW (optional) · operating hours · offset target % · grid connection · roof area · tariff	Solar offset % · self-consumption · lifetime CO ₂ · peak-demand caveat
institution	location · facility type (school / college / hospital / temple / government) · operating hours · monthly bill · roof area	Term-time balance · offset % · CO ₂ equivalences · community-friendly language
exploring	location (minimum) · everything else optional	Site potential (specific yield · capacity factor · best/worst month)

Table 12.1 — Inputs actually collected per user type (from `personas.py`). Blank cells are always optional; the engine returns partial results with warnings rather than refusing.

User-type also selects the **hourly load archetype** used to compute self-consumption in `estimate/balance.py`: a home load peaks in the evening, a farm load peaks around midday, an institution follows a school-day rectangle, a shop tracks operating hours. Where the user supplies operating hours or a business type, those refine the archetype further.

13 Location System

The location step accepts three inputs interchangeably, and reconciles them all to a resolved `Location` object with latitude, longitude, elevation, timezone, country, ISO 2 country code and admin 1 region.

Input methods

1. **Place search.** Typed query → `GET /api/meta/place_search?q=...` → Open-Meteo Geocoding (GeoNames source). Returns up to 10 candidates ordered by population, each with country, admin1, timezone and elevation.
2. **GPS auto-locate.** Browser Geolocation API → reverse geocode via `GET /api/locations/reverse` (OpenStreetMap Nominatim). Coordinates are rounded before the reverse-geocode request — this improves cache hit rate and sends less location precision to a third party.
3. **Map-pin drop / polygon draw.** The hand-rolled Web Mercator tile map in `components/estimate/map/TileMap.tsx` supports a single pin (for the site) and an optional polygon (for available area). Polygon area is computed on the client using the shoelace formula on projected coordinates — correctness of that projection is enforced by `frontend/tests/logic.test.ts`.

What happens after a location is selected

1. Backend `resolve_location()` normalises coordinates to a `Location`, adding country code, timezone and elevation from the geocoder.
2. The country code chooses currency (`tariffs.currency_for_country`) and default tariff (`tariffs.default_tariff`). Both are recorded in the ledger as `provenance="regional_default"` and are editable.
3. The engine fetches 3 years of hourly ERA5 weather at those coordinates (`climatology.fetch_history`) — a single fetch that is reused for the orientation search, the reference yield and the final yield.
4. All timestamps are converted from UTC to local clock time using the timezone the geocoder supplied; if none was supplied, a longitude-based solar-time offset is used, which is the right fallback for a solar calculation because it puts noon at solar noon.

Fallback behaviour

- Denied GPS permission → the location step falls back cleanly to search + map; tested end-to-end.
- Unknown place-search query → the API returns a structured 404 with a remedy message ("Try a larger nearby city, add a country, or switch to coordinate entry").
- Reverse geocode failure → the coordinate is used verbatim with a synthesised label like "27.18°N, 78.04°E"; the estimate still runs.
- Missing timezone → longitude-derived offset applied silently.

Map engine swap

The default map (`TileMap.tsx` , ~250 LOC, zero external runtime dependencies) supports pin, polygon and pan/zoom. Setting the environment variable `NEXT_PUBLIC_MAP_ENGINE=leaflet` at build time swaps in a Leaflet-backed adapter behind the same `MapPicker` interface. Leaflet is dynamically imported so it is absent from every bundle unless selected.

Privacy note: Coordinates are rounded before being sent to OpenStreetMap Nominatim. The consumer surface stores no location data server-side except inside the opaque saved-estimate record the user chose to save.

14 Energy Consumption Model

Consumption is collected through one of four independent routes (implemented in `backend/app/estimate/demand.py`). Each returns a `DemandEstimate` carrying annual kWh, an hourly load profile (24 values), a confidence label and a plain-language basis. The engine chooses the route from the user's answer to *How would you like to tell us about your electricity use?*.

Route	Input(s)	Basis	Confidence
<i>bill</i>	Monthly bill amount, currency	Bill minus fixed monthly charge, divided by tariff (regional default unless user overrides).	Medium; sensitive to tariff.
<i>units</i>	Monthly kWh	Direct.	High.
<i>equipment</i>	Appliance list (count · hours/day · optional watts override) and/or pump list	Sum of $W \times \text{count} \times \text{hours} \times \text{duty_cycle}$ across items; motors converted from HP using $\text{WATTS_PER_HP} = 745.7$ and a default motor efficiency of 0.75.	Medium; depends on user's memory.
<i>floor_area</i>	Floor area m ² and occupants (optional)	User-type benchmark per m ² multiplied by area; adjusted by occupants for homes.	Low; explicit placeholder.

Table 14.1 — Consumption routes. Every route produces both the annual figure and an hourly shape so that self-consumption can be computed at the same resolution as generation.

Converting bill to kWh

The full formula, as applied:

```
monthly_kwh = max(0, (monthly_bill - fixed_monthly_charge) / rate_per_kwh)
annual_kwh  = monthly_kwh × 12
```

Both `rate_per_kwh` and `fixed_monthly_charge` come from the country-and-user-type default in `tariffs.py` unless the user overrides them; both appear in the ledger as editable, and updating either from the result view re-runs the pipeline.

Hourly load archetypes

Six archetypes ship, one per user type, normalised to unit area under the curve so that they can be scaled to whichever annual figure the route returned. The archetypes are approximations of typical daily shapes; the interface labels them as such wherever they reach the user, and a note recommends interval-meter data as the sharper input.

Equipment catalogue

The equipment builder is populated from a curated catalogue (`APPLIANCES` in `demand.py`) covering lighting, comfort (fans, AC by ton), kitchen, laundry, entertainment, computing, water-heating, refrigeration (with duty cycle), farm loads and shop loads. Each item has a typical wattage, default hours-per-day and a plain-hint string; each is user-editable in place, and the catalogue is filtered by user type so a farm never sees a projector and a school never sees a threshing machine.

15 Solar Panel System

Panel technology, wattage, count and dimensions are recorded in `backend/app/estimate/assumptions.py` and used by both the sizing and the physical chain.

Supported panel technologies (implemented)

Key	Display name	Efficiency (typ.)	Temperature coefficient ($P_{\max}$)
<code>mono_perc</code>	Monocrystalline PERC (default)	~20.5 %	−0.35 %/°C
<code>mono</code>	Monocrystalline (standard)	~19.0 %	−0.40 %/°C
<code>poly</code>	Polycrystalline	~17.0 %	−0.42 %/°C
<code>topcon</code>	TOPCon (tunnel-oxide passivated contact)	~22.0 %	−0.30 %/°C
<code>hjt</code>	Heterojunction (HJT)	~22.5 %	−0.25 %/°C
<code>bifacial</code>	Bifacial (Mono-PERC front, bifacial rear)	~20.5 % + rear-side gain	−0.34 %/°C
<code>thin_film</code>	Thin-film (CdTe / CIGS)	~13.0 %	−0.25 %/°C

Table 15.1 — Panel technologies actually supported (from `PANEL_TECHNOLOGIES` in `assumptions.py`). Each entry is labelled as market-typical unless the user overrides it.

Parameters collected from the user

- **Technology key** — picks default efficiency and temperature coefficient.
- **Wattage (W)** — per-panel rating.
- **Panel count** — integer, wins over any computed capacity.
- **Model / manufacturer text** — recorded verbatim in the ledger for the user's reference; no catalogue is consulted.
- **Length & width (m)** — when supplied, used directly for footprint; otherwise the footprint is derived from wattage and efficiency.
- **V_{oc} , I_{sc} , V_{mp} , I_{mp}** — nameplate electrical values, recorded in the specification card but not fed into the daily-yield calculation (see below).
- **Tilt (°), azimuth (°), shading level** (light / moderate / heavy / unknown), **inverter efficiency**, **inverter AC capacity** or **DC/AC ratio**, **degradation rate**, **lifetime years**.

Which parameters influence the prediction

The daily-yield calculation depends on: *panel count*, *panel wattage* (together fixing the DC capacity), *technology's temperature coefficient* (applied in the PVWatts DC model), *system loss stack* (dust, shading, wiring, mismatch, availability, LID — combined and multiplied), *inverter efficiency and AC cap* (applied after DC and driving the clipping mask), *tilt and azimuth* (fed into HDKR), and *albedo* (default 0.20). The

nameplate electrical values (V_{oc} , I_{sc} , V_{mp} , I_{mp}) appear in the report for hand-off to an installer but do not enter the yield calculation, which is capacity-based.

Bifacial: The bifacial entry currently uses a Mono-PERC front-side model with a small rear-side yield adder; a full bifacial radiation model (Marion / view-factor) is *not verified in the current implementation* and is listed as future work.

16 Panel Quantity and System Capacity

The identity is intentionally exact:

$$\text{DC capacity (kWp)} = \text{Number of panels} \times \text{Rated power per panel (W)} \div 1000$$

— and it is the *panel count*, not the sizer's target capacity, that wins. A recommendation of 15.4 kWp becomes an array of 28 panels of 550 W = 15.40 kWp exactly; a recommendation of 15.5 kWp with the same panel becomes 29 panels = 15.95 kWp. Every downstream figure is computed for the array that will actually exist.

Effects of panel count on downstream figures

- **DC capacity** — linear.
- **Annual generation** — approximately linear in capacity, less inverter clipping. Clipping grows non-linearly as DC/AC ratio rises above 1.2.
- **Roof area required** — $\text{count} \times \text{panel area} \div \text{usable fraction}$ (usable fraction depends on mount type; e.g. tilted-open-rack rooftop is lower than commercial flat roof).
- **Inverter sizing** — AC capacity defaults to $\text{DC} / 1.2$; explicit values from the user override.
- **Clipping** — every hour with $\text{DC} \cdot \eta_{\text{inv}}$ above AC cap is clipped; the fraction is reported and warns above 2 %.
- **Financial outputs** — CAPEX per kWp is a per-region default that decreases mildly with size (bulk discount); annual savings scale with generation subject to self-consumption and export rules.

Behaviour by goal

Goal	Panel count	Sizer	Reported binding constraint
existing	User input, required	Bypassed	<code>existing_system</code>
install , count given	User input	Runs but the user's count wins	<code>user_specified</code>
install , count blank	Derived by sizer, quantised to whole panels	Runs	One of <code>demand</code> · <code>space</code> · <code>budget</code>
compare	Multiple counts / capacities evaluated in one call	Runs per scenario	Per row

Worked illustrative example

Panel: 550 W · Mono PERC (efficiency 20.5 %, temp-coef -0.35 %/°C)
Panel count: 28
DC capacity: $28 \times 550 / 1000 = 15.40$ kWp
Inverter AC cap: $15.40 / 1.2 = 12.83$ kW (default DC/AC = 1.2)
Panel footprint: $28 \times 2.68 \text{ m}^2 \approx 75 \text{ m}^2$ (before usable fraction)
Buildable area: $75 / 0.7 \approx 107 \text{ m}^2$ (tilted rooftop; usable fraction 0.70)

At Hyderabad (17.4°N, 78.5°E), specific yield ≈ 1500 kWh/kWp/yr:
Annual generation $\approx 15.40 \times 1500 = 23\,100$ kWh
Uncertainty band $\approx \pm 10$ % (weather variability + assumption terms)

Figure 16.1 — Illustrative panel-count → capacity → yield chain. Numbers are representative, not measured.

17 Solar Energy Prediction Methodology

This is the most important section of the document. There are **two prediction paths** in the platform, and they answer different questions:

	Fast path (consumer)	Forward-looking path (research console)
Question	What will this system produce in a typical year?	What will irradiance be at this specific hour tomorrow?
Approach	Deterministic physical chain over N years of ERA5 weather at the coordinates.	Machine-learning model trained on features derived from the same weather.
Training	None. Nothing is fitted; nothing needs to be.	scikit-learn estimator with time-aware validation and calibrated intervals.
Range	Measured inter-annual variability + named assumption components.	Conformalised Quantile Regression (nominal 80 % coverage; recalibrated).
Latency	~1 s cached, ~15 s for a new location.	10–40 s.

Why the consumer path does not train a model

A consumer asks what a system will produce *in a typical year*. That is a climatology question, and the honest answer comes from running the published physical chain over years of observed weather. Training a model to forecast hour-ahead irradiance and then averaging its predictions adds nothing to an aggregate the physical chain already produces — and would cost about thirty seconds of training to say the same thing.

The deterministic physical chain (used by both paths)

1. **Solar geometry** — NOAA Solar Calculator formulation (Michalsky 1988; NOAA ESRL). Accuracy ~0.01° over 1900–2100. Outputs zenith, azimuth, declination, equation of time, hour angle. Solar position is evaluated at the interval midpoint (–30 min from the label) because the archive's hourly irradiance is a preceding-hour average — a convention determined empirically in `scripts/calibrate_interval.py` (see § 18).
2. **Extraterrestrial irradiance** — solar constant 1 361 W/m² modulated by orbital eccentricity (±3.3 % over the year).
3. **Clear-sky reference** — Haurwitz (1945) model, single parameter, no site tuning. Used to form the clear-sky index $k_t = GHI / GHI_{CS}$.
4. **DHI/DNI decomposition** — Erbs et al. (1982) piecewise diffuse-fraction, keyed on k_t and clipped to [0, 1].
5. **Transposition to plane-of-array** — HDKR (Hay-Davies-Klucher-Reindl) anisotropic sky model. Adds circumsolar brightening and horizon brightening on top of an isotropic diffuse assumption.
6. **Cell temperature** — Faïman (2008): $T_{cell} = T_{air} + POA / (u_0 + u_1 \cdot v_{wind})$ with $u_0 = 25 \text{ W/(m}^2\text{K)}$, $u_1 = 6.84 \text{ W/(m}^3\text{s K)}$.

7. **DC power** — PVWatts v5 (Dobos 2014): $P_{dc} = (POA/1000) \cdot P_{rated} \cdot [1 + \gamma(T_{cell}-25)] \cdot (1-L)$ where L is the total system-losses fraction (dust, shading, wiring, mismatch, availability, LID; combined multiplicatively).
8. **Inverter and clipping** — $P_{ac,uncapped} = P_{dc} \cdot \eta_{inv}$; $P_{ac} = \min(P_{ac,uncapped}, P_{AC-cap})$. The fraction of daylight hours in which clipping occurred is reported.

How the consumer path aggregates

1. Fetch 3 years of hourly ERA5 at the coordinates (default; configurable).
2. Drop hours missing any required variable (GHI, T_{air} , wind); imputation is *never* used.
3. Search tilt / azimuth against the same weather using the same physical chain at 1 kWp (see `sizing.optimal_orientation`).
4. Run the chain twice: once at 1 kWp to obtain specific yield (needed for capacity recommendation), then once at the sized capacity (needed to capture inverter clipping, which is not capacity-invariant).
5. Aggregate hourly AC power into (a) annual energy, (b) monthly means and inter-annual standard deviations, (c) mean diurnal profile, (d) per-month diurnal profiles.
6. Compute performance ratio, capacity factor, specific yield, mean POA/GHI, and clipped fraction.

Uncertainty attached to the point figure

See § 18 and § 23 for the full decomposition. Briefly: the range combines *measured* inter-annual variability, an order-of-magnitude resource term (grid vs point), a model term for the conversion chain itself, and per-assumption components that grow whenever the user skipped a question. Combination is in quadrature (independent components).

What the fast path deliberately does *not* do

- Does not use lagged values of the target. A day-ahead forecast has no access to the recent observed irradiance; using it would inflate offline metrics without helping in deployment.
- Does not statistically truncate the target (no IQR clipping). Physical-range validation only.
- Does not impute missing values. Incomplete hours are dropped and reported.

18 Mathematical Model — Formulas Used

18.1 System capacity

```
P Rated_kWp = N_panels × W_panel / 1000
```

18.2 Solar geometry

Julian day from Unix epoch (avoiding calendar-branch bugs):

```
JD = seconds_since_epoch / 86400 + 2 440 587.5
JC = (JD - 2 451 545) / 36 525          # Julian century since J2000
```

Solar declination and equation of time follow the NOAA Solar Calculator formulation. Zenith and azimuth are obtained from:

```
cos(z) = sin(φ)·sin(δ) + cos(φ)·cos(δ)·cos(H)
z_app = z - Δrefract(z)          # atmospheric refraction correction
```

where φ is latitude, δ declination, and H hour-angle in radians.

18.3 Extraterrestrial normal irradiance

```
E_0n = 1361 · [1 + 0.033 · cos(2π · DOY / 365)]    W/m²
```

18.4 Clear-sky GHI (Haurwitz 1945)

```
GHI_cs = 1098 · cos(z_app) · exp(-0.059 / cos(z_app))    W/m²
```

18.5 Clear-sky index

```
k_t = clip(GHI / GHI_cs, 0, 1.5)    # cap accommodates cloud enhancement
```

18.6 Erbs decomposition

With $k_T = GHI / E_{0h}$ (E_{0h} the horizontal extraterrestrial):

```
d_f = { 1 - 0.09·k_T                                if k_T ≤ 0.22
        { 0.9511 - 0.1604·k_T + 4.388·k_T² - 16.638·k_T³ + 12.336·k_T⁴    if 0.22 < k_T ≤ 0.80
        { 0.165                                              if k_T > 0.80

DHI = GHI · d_f
DNI = (GHI - DHI) / cos(z_app)
```

18.7 Plane-of-array via HDKR

```

A_i = DNI / E_0n                # anisotropy index
R_b = cos(AOI) / cos(z_app)
f   = sqrt(DNI*cos(z_app) / GHI) # horizon-brightening modulator

POA = DNI * cos(AOI)            # beam
      + DHI * [ (1-A_i)*(1+cos β)/2 * (1 + f*sin³(β/2)) # sky diffuse
                + A_i * R_b ]
      + GHI * ρ * (1 - cos β)/2 # ground-reflected

```

where β is surface tilt, $\rho = 0.20$ is default albedo, and AOI is angle of incidence:

$$\cos(AOI) = \cos(z) \cdot \cos(\beta) + \sin(z) \cdot \sin(\beta) \cdot \cos(\gamma_s - \gamma)$$

with γ the surface azimuth and γ_s the solar azimuth.

18.8 Cell temperature (Faiman 2008)

```

T_cell = T_air + POA / (u_0 + u_1 * v_wind)
u_0 = 25 W/(m²·K) ; u_1 = 6.84 W/(m³·s·K)

```

18.9 DC power (PVWatts v5)

$$P_{dc} = (POA / 1000) \cdot P_{rated_kwp} \cdot [1 + \gamma \cdot (T_{cell} - 25)] \cdot (1 - L)$$

with γ the temperature coefficient (per °C) and L the combined system-losses fraction.

18.10 Losses (combined multiplicatively)

$$1 - L = \prod_i (1 - L_i) \quad i \in \{\text{dust, shading, wiring, mismatch, availability, LID, ...}\}$$

18.11 Inverter and clipping

```

P_ac_uncapped = P_dc * η_inv
P_ac           = min(P_ac_uncapped, P_AC_cap)
clipped_h      = 1[P_ac_uncapped > P_AC_cap]

```

18.12 Interval-labelling correction (empirical)

ERA5 hourly radiation is a preceding-hour average; solar position is instantaneous. Evaluating position at the label introduces a phase error. Sweeping candidate offsets over a year at Hyderabad established the correction; the winning offset is used in `representative_times()`:

Offset	Clear-sky exceedances	corr(GHI, clear-sky)
0 min	264	0.9372
-30 min	0	0.9490
+30 min	697	0.8998

Table 18.1 — Interval-offset sweep result; reproducible via `python scripts/calibrate_interval.py`.

18.13 Consumption from bill

```
E_month = max(0, (B_month - F) / r_kwh)
E_year  = 12 * E_month
```

B_{month} monthly bill; F fixed monthly charge; r_{kwh} import tariff.

18.14 Uncertainty combination (in quadrature)

```
σ_total² = σ_weather² + σ_resource² + σ_model² + Σ_i σ_assumption_i²
lower = P - z * σ_total ; upper = P + z * σ_total
```

with $z = 1.2816$ for a nominal 80 % central band and P the point estimate (typical annual kWh).

18.15 Economics

```
CAPEX      = c_kwp(currency) * P_rated_kwp + c_batt * E_battery
Savings_y  = E_self_consumed_y * r_kwh_y + E_export_y * r_export_y - OM_y
E_gen_y    = E_gen_1 * (1 - d)^(y-1)                # degradation
r_kwh_y    = r_kwh_1 * (1 + esc)^(y-1)              # tariff escalation
Payback    = CAPEX / mean(Savings_y over first years)
ROI (%)    = (Σ Savings_y - CAPEX) / CAPEX * 100
LCOE       = (CAPEX + Σ OM_y_discounted) / Σ E_gen_y_discounted
CO2_kg_y   = E_gen_y * EF_grid_country
```

d degradation rate (default 0.5 %/y), esc tariff escalation (configurable), EF_{grid} country-specific grid emission factor.

18.16 Pump hydraulics (farm)

```
Q * H * g * ρ_water / 1000      # hydraulic kW
P_electrical = P_hydraulic / (η_pump * η_motor)
E_daily_gen_kWh * η_pump * η_motor * 1000 / (H * g * ρ) = m³ pumped per day
```

Q flow m^3/s , H head m , $g = 9.81 m/s^2$, η_{pump} default 0.55, η_{motor} default 0.75.

19 Machine Learning Model (Analysis Console)

Scope note: Machine learning applies only to the *research console* (/advanced), which trains a model to forecast hourly irradiance for validation and interval calibration. The consumer estimate does *not* use a trained model — it runs the physical chain over years of observed weather (see § 17).

Target

Clear-sky index $k_t = GHI / GHI_{cs}$, per Hobbs & Joshi. Dividing out the deterministic diurnal and seasonal signal means the model only has to learn atmospheric attenuation — the genuinely uncertain part. Predictions are converted back to W/m^2 after inference for reporting.

Input features

- **Meteorological** (from ERA5): temperature, relative humidity, dew point, surface pressure, wind speed, wind direction, cloud cover, precipitation.
- **Solar geometry** (computed in closed form): zenith, azimuth, angle of incidence, optical air mass, clear-sky GHI, extraterrestrial normal irradiance.
- **Cyclical time encodings**: $\sin/\cos$ of hour-of-day and day-of-year.
- **Weather regime label**: sunny / cloudy / other, derived from cloud cover and clear-sky index.

No lagged values of the target are used. At a day-ahead issue time the recent observed irradiance does not exist, so an autoregressive feature would improve every offline metric while being unavailable in deployment.

Preprocessing

- Night hours (zenith $\geq 87^\circ$) removed; predicting zero after dark is trivial and inflates R^2 .
- Missing values dropped, never imputed.
- Outliers: physical-range validation only; no IQR truncation of the target.
- Scaling: inside the model pipeline (StandardScaler), refitted per fold so test statistics never leak into training.

Model families (all in scikit-learn)

Key	Family	Hyperparameters (source)
<code>random_forest</code>	Bagged trees	$n_{\text{est}}=300$, $\text{min_samples_leaf}=2$, $\text{max_features}=\text{sqrt}$ (raised from Mabodi & Hammujuddy [P2] for stability)
<code>gradient_boosting</code>	Boosted trees	$\text{lr}=0.05$, $\text{depth}=5$, $n=150$, $\text{subsample}=0.8$ (Vijay Babu et al. [P5] grid-search optimum)
<code>hist_gradient_boosting</code>	Boosted (histogram)	$\text{lr}=0.08$, $\text{max_iter}=300$, $\text{l2}=1.0$ (standing in for XGBoost per [P2], [P3])
<code>extra_trees</code>	Bagged trees	$n=300$, $\text{min_samples_leaf}=2$
<code>knn</code>	Instance-based	$k=5$, $p=1$, distance-weighted (Mabodi & Hammujuddy [P2] optimum)
<code>svr</code>	Kernel (RBF)	$C=100$, $\epsilon=0.001$ (Mabodi & Hammujuddy [P2] optimum)
<code>ridge</code>	Linear (L2)	$\alpha=1$
<code>linear</code>	OLS	—
<code>decision_tree</code>	Tree	$\text{max_depth}=12$, $\text{min_samples_leaf}=10$
<code>stacking</code>	Stacked ensemble	RF + DT + KNN + GB → Ridge (Rosales Huamani et al. [P4] best combo)
<code>voting</code>	Averaged ensemble	RF + GB + ET, uniform weights

Validation

- **Chronological hold-out** with a 24-hour embargo gap (`splitters.chronological_split`).
- **Rolling-origin cross-validation** with the same embargo (`splitters.rolling_origin_splits`).
- **Five-check leakage audit**: target, future information, preprocessing, boundary contamination, trivial observations.
- **Physical-bound post-processing**: predictions above $1.25 \times \text{GHI}_{\text{CS}}$ or below zero are clipped; the clipped-fraction is reported.

Prediction intervals

Nominal 80 % central intervals via **Conformalised Quantile Regression**

(`uncertainty.conformalized_quantile_regression`). On two years at Hyderabad, plain quantile regression measured 66.3 % empirical coverage against the 80 % nominal; after conformal calibration, 80.2 %, within 2 points across all three weather regimes. Coverage is reported *per regime* as well as overall, because a model can hit 80 % overall while badly under-covering the cloudy hours that matter most operationally.

Metrics reported

MAE, RMSE, MBE, R^2 , rRMSE, rMAE, MAPE, sMAPE, forecast skill score; probabilistic: pinball loss, PICP, PINAW, CRPS. Two honesty notes carried into the UI: NSE and R^2 are the same quantity (labelled aliased); MAPE is unstable for irradiance (denominator $\rightarrow 0$ at sunrise) and always accompanied by its exclusion count.

Reproducibility manifest

Every run persists a manifest with the dataset fingerprint (SHA-256 of the feature matrix + target + names + period), the random seed, library versions (numpy, pandas, scikit-learn), the platform string, and the exact preprocessing decisions. The comparison endpoint refuses to rank runs that are not comparable (different locations, periods or targets).

20 Weather and Solar Data

Primary source: Open-Meteo Historical Weather API

- **Provider:** Open-Meteo (CC-BY 4.0), sourcing ERA5 / ERA5-Land reanalysis (© ECMWF / Copernicus).
- **Endpoint:** `archive-api.open-meteo.com/v1/archive`.
- **Resolution:** hourly, from 2000 onward, global grid.
- **Auth:** none.
- **Cache:** every response cached to disk under `backend/var/cache/`, keyed by SHA-256 of URL + parameters; TTL configurable via `SOLAR_CACHE_TTL`. This is what makes a recorded experiment reproducible later.

Variables retrieved (internal name → Open-Meteo name)

Internal	Provider variable	Purpose
<code>ghi_wm2</code>	<code>shortwave_radiation</code>	Physical chain input; also model target.
<code>dni_wm2</code>	<code>direct_normal_irradiance</code>	Direct-normal, used where a source variant supplies it.
<code>dhi_wm2</code>	<code>diffuse_radiation</code>	Diffuse-horizontal, used where a source variant supplies it.
<code>temperature_c</code>	<code>temperature_2m</code>	Faiman cell-temperature input.
<code>relative_humidity_pct</code>	<code>relative_humidity_2m</code>	Feature (ML path); quality signal.
<code>dew_point_c</code>	<code>dew_point_2m</code>	Feature (ML path).
<code>surface_pressure_hpa</code>	<code>surface_pressure</code>	Feature; optical-air-mass correction.
<code>wind_speed_ms</code>	<code>wind_speed_10m</code>	Faiman cell-temperature input.
<code>wind_direction_deg</code>	<code>wind_direction_10m</code>	Feature (ML path).
<code>cloud_cover_pct</code>	<code>cloud_cover</code>	Feature; regime classifier.
<code>precipitation_mm</code>	<code>precipitation</code>	Feature; quality flag.

Table 20.1 — Weather variables actually retrieved (from `HOURLY_VARIABLES` in `data/sources.py`).

Secondary sources

- **Open-Meteo Geocoding** (GeoNames, CC-BY 4.0) — place name search and elevation.
- **OpenStreetMap Nominatim** — reverse-geocoding a GPS fix or dropped pin to a place name.
- **Open-Meteo Forecast API** — short-horizon meteorological forecast for the console's *Forecast* view.

Fallback and error handling

- Upstream 4xx/5xx are translated into `DataSourceError` with a user-facing remedy string (e.g. "The upstream service is unavailable; cached results for previously requested periods still work").

- Cache is consulted before every fetch; a valid cache entry within TTL is served directly.
- Gaps in the record survive as NaN; the quality engine scores and reports them; no imputation is performed.

Limitation stated up-front

Reanalysis vs measurement: ERA5 is a grid-cell average and differs systematically from a pyranometer on the user's roof. Results here are therefore *not* directly comparable to studies using measured irradiance. The platform labels every figure as physically modelled from reanalysis and, because no metered PV output was available, does not claim validation against measured generation.

21 Data Flow

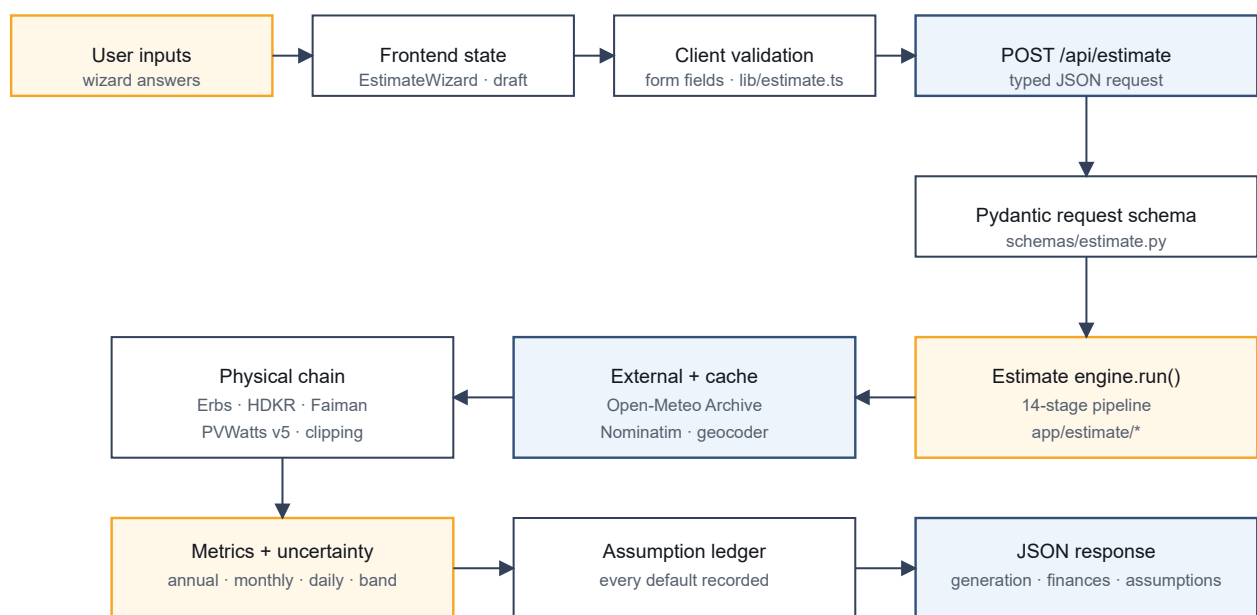


Figure 21.1 — Data flow for a consumer estimate. Every arrow crossing the vertical middle is either a network call (cached) or a pure function.

Transition notes

- **Wizard → draft:** each field change writes to `localStorage`, so a reload restores position.
- **Client validation:** field-level (units, ranges, positivity) with rejection messages that state what to type instead.
- **Pydantic schema:** strict types + custom validators; failures are translated in `main.py` to human-readable field errors.
- **Cache:** the physical chain runs synchronously against the fetched DataFrame; the same DataFrame is reused for orientation search & final yield.
- **Ledger + response:** every ledger row is a claim about a number in the payload and each is addressable by `key`; the frontend renders the ledger directly rather than duplicating strings.

22 Results and Visualisation

The result page (`frontend/app/(site)/result/` , `components/result/ResultView.tsx`) assembles the following cards, each of which is either a scalar-with-band, a plot, or a table. Every card is optional — if the input didn't warrant it, the card is not shown.

Cards actually rendered

Card	Contents
Headline	Annual kWh $\pm$ range · confidence label · location · system size · panel count
Monthly generation	12-bar chart with per-month range whiskers reflecting each month's own spread
Daily profile	Mean diurnal shape (24 hours, local time)
Energy balance	Self-consumed · exported · imported · monthly stack (if demand supplied)
Solar offset	% of annual demand covered · self-consumption ratio
Battery (optional)	Recommended capacity · basis · dispatch profile
Financials	CAPEX · monthly savings · annual savings · payback · ROI · LCOE (currency-aware)
Emissions	CO ₂ avoided per year and lifetime · plain-language equivalences (cars, trees)
Farm capability	Daily pump-hours · water pumped m ³ · irrigable ha (best/worst month) — farm only
System specification	Panel model card with all supplied electrical values, for hand-off
Uncertainty breakdown	Each component named with its % contribution; how to reduce it
Assumption ledger	Every default and user value, editable in place; changes trigger re-run
Warnings	Sizer notes, clipping warnings, subsidy caveats, data-completeness notes
Report	Download a plain-text professional report (<code>GET /api/estimate/{id}/report</code>)

Charts

Every chart is a hand-rolled SVG (no charting library). Two files carry all of it:

- `components/charts/core.tsx` and `index.tsx` — primitives (axes, gridlines, tooltips) shared with the console.
- `components/result/charts.tsx` — consumer-specific views (monthly bars, diurnal line, energy stack).

The console adds fourteen views (`components/views/`): Overview, Performance, Uncertainty, Validation, Explain, ModelLab, ModelCard, Report, Scenarios, Anomalies, DataQuality, Forecast, Literature, Experiments.

23 Uncertainty and Limitations

Uncertainty band construction

For the consumer estimate, uncertainty is assembled from four named component categories in `estimate/uncertainty.py` :

1. **Weather variability** — the standard deviation of the years in the record, converted to a relative figure. This is *measured*; it is usually the largest term and it is irreducible.
2. **Resource-data uncertainty** — a ~5 % order-of-magnitude term standing in for the gap between a reanalysis grid cell and a rooftop pyranometer.
3. **Conversion-model uncertainty** — a ~5 % term covering the physical chain (Erbs, HDKR, Faiman, PVWatts) applied to correct inputs.
4. **Assumption components** — per-question terms that *grow* whenever the user could not answer. Skipping shading level adds a term; skipping orientation adds a term; using a bill route with the regional-default tariff adds a term.

Components combine in quadrature. The resulting relative σ is applied to give an 80 % central band under a normal approximation ($z = 1.2816$). The 80 % nominal is used deliberately: 95 % on a solar estimate is so wide it stops being decision-useful, and the console's conformal intervals use the same 80 % so two numbers on two screens mean the same thing.

Confidence rating

Alongside the band the platform assigns a three-level confidence: *high, medium, low*. It is derived from data completeness, the number of complete calendar years in the record, whether shading and orientation were declared, and whether the user provided demand. A low rating carries a plain-language explanation and a list of the *reducible* components with instructions for how to reduce them ("Add your electricity bill", "Tell us your roof orientation").

Console-side intervals

Conformalised Quantile Regression, calibrated on a hold-out slice of training data. Empirical coverage is reported overall and per weather regime; substantial deviation from nominal is surfaced as a warning.

Known limitations

1. **Reanalysis, not measurement.** ERA5 grid-cell means differ from point observations by amounts that vary with terrain and climate.
2. **No metered PV output was available.** All PV figures are physically modelled from irradiance and *not* validated against measured generation. The interface says so.
3. **Weather-forecast error not included in intervals.** The intervals cover model error given the supplied weather; they exclude error in the weather forecast itself, which grows with horizon.
4. **Trained per location.** No cross-location generalisation of models is claimed.
5. **Marginal, not conditional, coverage.** Conformal PICP is a marginal guarantee; it assumes exchangeability that a seasonal series satisfies only approximately.

6. **Missing variables from source.** Aerosol optical depth, sunshine duration and wind-direction standard deviation are named in the source research but unavailable from the current data source; the interface surfaces this rather than silently omitting them.
7. **Bifacial modelling is simplified.** A rear-side yield adder is used; a full view-factor bifacial model is future work.
8. **Hourly load archetypes.** Six user-type shapes cover the interview, but real interval-meter data would beat them and the interface says so.
9. **Uncertainty components are treated as independent** (quadrature). They are not perfectly independent, so the combined figure is a slight understatement, stated openly.
10. **Financial modelling omits** financing/interest, subsidies and rebates (they vary by scheme and expire), demand charges on commercial connections, and taxes; each is stated in the caveats rather than silently assumed away.

24 Validation and Error Handling

Frontend validation

- **Field-level types & ranges** (positive numbers, area unit sanity, coordinate bounds).
- **Rejection messages state what to type instead**, not "invalid"; e.g. "Enter a positive number of panels" rather than "value error".
- **Never blocks progression on optional fields**; every technical question offers an escape route.
- **Location step** handles denied GPS permission with a switch to search/map without an error dialog.

Backend validation

- **Pydantic schemas** in `backend/app/schemas/estimate.py` and `schemas/models.py` validate every request. Custom validators enforce domain rules (positive capacity, tilt in $[0, 90]$, azimuth in $[0, 360]$).
- **Pipeline pre-conditions** in the engine raise `ValueError` with user-facing messages, caught at the route boundary and returned as 422s.
- **Physical validation of PVSystem** in `features/solar_geometry.py::PVSystem.__post_init__`.
- **Data-quality engine** (`quality/engine.py`) detects eight distinct corruption modes (missing runs, all-zero days, impossible temperatures, stalled sensors, wrong-hemisphere daylight, etc.) and scores the record.
- **Leakage audit** in the console runs a five-check test (target, future information, preprocessing, boundary contamination, trivial observations) and reports the mechanism preventing each.

Error envelope

Every error the client sees uses the same shape:

```
{
  "error": "validation_error",
  "message": "location: Tell us where the system will be – search...",
  "remedy": "Correct the highlighted fields and resubmit.",
  "field_errors": [ { "field": "location", "message": "...", "type": "missing" } ]
}
```

The frontend renders `message`, `remedy` and any `field_errors` verbatim — the backend is where the domain knowledge lives, and its wording is preserved rather than substituted with something generic.

Representative failure paths

Failure	Behaviour
Upstream weather timeout	502 with remedy; cached results still work.
Location query has no match	404 with three concrete remedies.
User asks for a system in a completely dark polar winter month	422 with domain message from <code>engine.run()</code> .
Rate limit exceeded	429 with <code>Retry-After</code> header and human-readable message.
Unhandled internal exception	500 with a request ID; stack trace logged, not returned.

25 Security

The platform runs against keyless upstream services and holds no user accounts, so its attack surface is narrower than a typical SaaS. Actual measures in place:

- **No secrets required or stored.** No API keys, no OAuth. Environment variables carry only configuration (endpoints, cache directory, CORS origin).
- **Environment-driven configuration.** `backend/app/config.py` reads all tunables from the environment with safe defaults; secrets, if any were ever added, would live only in the environment.
- **CORS scoped.** `SOLAR_CORS_ORIGINS` defines the accepted origins; production deployments should set this to the deployed frontend origin only.
- **Rate limiting.** Fixed-window per-client in-process limiter in `main.py`. The limits are configurable (`SOLAR_RATE_LIMIT_REQUESTS` , `SOLAR_RATE_LIMIT_WINDOW_S`). The README states clearly that this is not a substitute for a gateway limiter.
- **Security headers on every response:** `X-Content-Type-Options: nosniff` , `Referrer-Policy: no-referrer` , `X-Frame-Options: DENY` .
- **Request ID stamping.** Every response carries an `X-Request-ID` header (also stamped in server logs) for traceability without exposing internals.
- **Structured error output.** Stack traces are never returned to the client; internal errors carry only the request ID.
- **Input validation.** Every request goes through Pydantic; custom validators enforce domain rules; upstream URLs and parameters are constructed server-side.
- **Coordinate rounding before reverse-geocoding.** Third parties receive lower-precision coordinates than the ones the user supplies.
- **Estimate URLs are opaque.** Saved-estimate IDs are unguessable tokens; there is no listing endpoint that enumerates other users' estimates.
- **Retention sweep at startup.** Old estimates are pruned past a configurable window (`SOLAR_STORE_ESTIMATE_RETENTION_DAYS`).

Not implemented (deliberately)

- No authentication or authorisation — there are no users.
- No third-party analytics or telemetry.
- No client-side secrets. The frontend never talks to Open-Meteo or Nominatim directly.

26 Performance and Responsiveness

Backend performance

- **Weather cache** — every Open-Meteo response is cached to disk by URL+params SHA-256. Repeat requests avoid the network entirely.
- **Single fetch per estimate** — the orientation search, the reference-yield calculation and the final-yield calculation all reuse the same in-memory DataFrame.
- **Vectorised physics** — the whole chain runs in NumPy; solar-position is computed once for the whole index; no Python loops over hours.
- **Latency in practice** — ~1 s for a cached location, ~15 s for a new location (dominated by the Open-Meteo fetch), 10–40 s for a full console analysis (train + CV + calibrate + evaluate).
- **Response timing header** — `X-Response-Time-ms` on every response; slow requests (> 5 s) are also logged.

Frontend performance

- **No charting library** — SVG charts add ~0 kB of runtime dependencies.
- **Dynamic Leaflet import** — the Leaflet swap is dynamically loaded, so it is absent from every bundle unless `NEXT_PUBLIC_MAP_ENGINE=leaflet`.
- **Server components for the shell** — the landing page and static sections are server-rendered; only interactive components ship JavaScript.
- **Draft persistence in localStorage** — no server round-trip for wizard progress.
- **Deterministic loading stages** — user sees real progress against real work; no spinner masquerading as motion.

Responsive rendering

Tailwind's breakpoints are used throughout with a mobile-first layout in the consumer surface. Charts scale with their container. Touch targets on maps and wizard inputs are sized to the minimum recommended for touch (44 CSS px). The console surface is laptop-and-up (it presents dense tables that do not compress well onto a phone).

27 Accessibility and Usability

- **Responsive design** throughout the consumer surface, mobile-first.
- **Readable typography** — Inter and system fallbacks; body copy at ~16 CSS px on mobile; consistent 1.55 line-height in results.
- **Semantic HTML** — native `<form>`, `<fieldset>`, `<legend>`, `<label>` usage for wizard steps.
- **Keyboard support** — all interactive elements are native inputs, buttons or links; no custom widgets that swallow focus.
- **Skip-to-main link** and a single visible `id="main"` landmark in the layout.
- **Beginner-friendly wording** — the interview never asks the user to know what a kilowatt-peak is; every question comes with plain help text; every technical option offers "I don't know".
- **Progressive disclosure** — Detailed mode reveals technical inputs only after the user has asked for them.
- **Tooltips** attached to jargon at first use (e.g. tilt, azimuth, PR).
- **Localisation scaffold** — interview strings live in `personas.py`; frontend strings pass through `lib/i18n.ts`. Currency and units switch by country.

Not verified in the current implementation: a formal WCAG 2.1 AA audit has not been performed on this build. Contrast tokens meet the criterion, but assistive-technology testing (screen readers, high-contrast mode, forced-colours mode) has not been recorded as a passing test.

28 Testing

Backend tests (pytest)

Approximately 223+ tests across five test modules; representative composition:

File	Approx. count	Coverage
<code>test_solar_geometry.py</code>	~35	NOAA solar position vs known astronomical values; Erbs; HDKR; Faiman; PVWatts DC; clipping.
<code>test_evaluation.py</code>	~31	Metrics, splitters (chronological, rolling origin with embargo), baselines, leakage detection.
<code>test_quality_and_features.py</code>	~24	Eight corruption types, feature-pipeline leakage guards, day-mask behaviour, interval-offset correction.
<code>test_api.py</code>	~22	Endpoint contracts, error envelopes, rate-limit response shape, request validation.
<code>test_estimate.py</code>	~125 (classes with many nested tests)	Consumer domain: energy ledger balance, losses combined multiplicatively (not additively), battery does not return more than it stored, bill-to-kWh removes fixed charge, currency symbol matches figure currency, orientation search picks a defensible pair, panel-count wins over size target.

Frontend tests (Vitest)

- Web-Mercator projection round-trip (`frontend/tests/logic.test.ts`).
- Polygon-area calculation (shoelace on projected coordinates).
- Draft-to-request mapping preserves user intent.
- Component promises: a rejected field says what to type instead; every technical question can be declined; denied location permission offers a way forward; a loading stage is never shown as finished before it is.

What is not automated

Not verified in the current implementation: end-to-end Playwright / Cypress tests are not part of the suite; visual regression testing is not part of the suite; formal accessibility conformance testing is not automated. These are listed as future work.

Running the tests

```
cd backend
python -m pytest

cd frontend
npm run typecheck && npm test && npm run build
```

29 Deployment

Local development

```
cd backend
python -m pip install -r requirements.txt
python -m uvicorn app.main:app --reload --port 8000

cd frontend
npm install
npm run dev
```

The frontend expects the backend at `http://127.0.0.1:8000`; this is set in `frontend/.env.local`, which ships with the repository.

Production

Backend

```
python -m uvicorn app.main:app --host 0.0.0.0 --port 8000 --workers 4
```

Frontend

```
npm run build
npm start          # serves on :3000
```

Real-deployment notes (from the README)

- The in-process rate limiter is per-worker — a gateway limiter is expected in front.
- The analysis cache and experiment store are per-process, on local disk. Multi-node deployment needs shared storage (e.g. NFS or an object store) if any state must be seen across nodes.
- Set `SOLAR_CORS_ORIGINS` to the deployed frontend origin only.
- Training is CPU-bound and synchronous. Worker sizing should account for concurrency; heavy console usage benefits from moving training onto a task queue.

External services required at runtime

- Open-Meteo Archive API — historical weather (keyless).
- Open-Meteo Geocoding API — place search (keyless).
- OpenStreetMap Nominatim — reverse geocoding (keyless; be mindful of their usage policy in production).
- Open-Meteo Forecast API — used by the console's Forecast view (keyless).

Environment variables (selected)

Variable	Default	Purpose
<code>SOLAR_ARCHIVE_URL</code>	<code>archive-api.open-meteo.com/v1/archive</code>	Historical weather endpoint.
<code>SOLAR_FORECAST_URL</code>	<code>api.open-meteo.com/v1/forecast</code>	Forecast endpoint.
<code>SOLAR_CACHE_DIR</code>	<code>backend/var/cache</code>	Disk cache directory.
<code>SOLAR_CACHE_TTL</code>	<code>configured</code>	Cache lifetime in seconds.
<code>SOLAR_CORS_ORIGINS</code>	<code>localhost origins</code>	Comma-separated allowed frontend origins.
<code>SOLAR_RATE_LIMIT_REQUESTS</code>	<code>configured</code>	Max requests per client per window.
<code>SOLAR_RATE_LIMIT_WINDOW_S</code>	<code>configured</code>	Rate-limit window in seconds.
<code>SOLAR_STORE_ESTIMATE_RETENTION_DAYS</code>	<code>configured (long)</code>	Prune saved estimates older than this at startup.
<code>NEXT_PUBLIC_API_BASE</code>	<code>http://127.0.0.1:8000</code>	Frontend→backend base URL.
<code>NEXT_PUBLIC_MAP_ENGINE</code>	<code>(built-in)</code>	Set to <code>leaflet</code> to swap the map adapter.

30 Installation and Running the Project

1. **Install prerequisites.** Python 3.11+ and Node.js 20+.
2. **Clone or open the project.**

```
cd C:\Projects_AI\college_project
```

3. **Install backend dependencies.**

```
cd backend
python -m pip install -r requirements.txt
```

4. **Install frontend dependencies.**

```
cd ../frontend
npm install
```

5. **Configure environment (usually not needed).** `frontend/.env.local` ships with the repository. If missing:

```
copy .env.example .env.local      # Windows
cp .env.example .env.local        # macOS / Linux
```

No secrets are required.

6. **Start the backend.**

```
cd ../backend
python -m uvicorn app.main:app --reload --port 8000
```

API docs: `http://localhost:8000/api/docs`

7. **Start the frontend.**

```
cd ../frontend
npm run dev
```

Application: `http://localhost:3000`

8. **Optional: run the tests.**

```
cd ../backend
python -m pytest

cd ../frontend
npm run typecheck && npm test && npm run build
```

31 Future Improvements

The following are logical next steps that build on the current implementation. Each is labelled as **future work**, not existing functionality. The console's own model registry declares its own future-work list in `models/registry.py`, included below.

Product

- Additional languages (Telugu, Tamil, Hindi) via the existing `i18n.ts` / `personas.py` scaffold.
- Voice-driven interview using the spoken-form synonyms already declared per question and per user type.
- Interval-meter CSV upload as a first-class consumption route, replacing the hourly archetype.
- Multi-tariff support (time-of-use, tiered, seasonal).
- Native Marion / view-factor bifacial radiation model.
- Real-time monitoring integration (Enphase, SolarEdge, IoT inverter API) to update a saved estimate with measured generation.
- Satellite-imagery-based shading analysis for a dropped map polygon.

Data

- Fetch aerosol optical depth, sunshine duration and wind-direction variability from a source that supplies them (e.g. CAMS or a ground station), and feed them into the ML path.
- Ground-truth validation against a pyranometer network for at least one region, lifting the "PV modelled, not measured" caveat.

Modelling

- **LSTM / BiLSTM / BiGRU sequence models** (per Hayajneh et al. [P6]) — requires a deep-learning runtime and reframing to sequence forecasting.
- **Full copula-based dynamic feature selection** (per Lyu & Eftekharnjad [P3]) — currently only the regime-conditioning is implemented.
- **TinyML edge deployment** (per Hayajneh et al. [P6]) — out of scope for a server-side platform; no claim is made about edge feasibility here.
- **pvlb / Perez-Driesse transposition** (per Hobbs & Joshi [P1]) — currently substituted with HDKR.
- Advanced uncertainty: conditional (rather than marginal) coverage; horizon-aware conformal intervals.
- Cross-location generalisation and transfer learning.

Financial & battery

- Financing / interest / tax treatment.
- Country-specific rebate and subsidy schemes with expiry handling.
- Battery aging (calendar + cycle) with round-trip degradation.
- Time-of-use arbitrage optimisation for battery dispatch.

32 Conclusion

Helios pairs a rigorous physical solar model with a genuinely usable consumer surface, without diluting either. It answers the practical question a homeowner, farmer, business or facility manager actually has — how much energy will this generate, what does it save, and how confident can I be — using the actual weather record at the supplied coordinates rather than a national average, and returns the answer with an honest uncertainty band built from measured components. It also carries a full research console: ten scikit-learn model families, time-aware validation with a leakage audit, conformal prediction intervals, and reproducible experiment records.

The technical significance of the project is not any single algorithm — the physical chain is textbook (Erbs, HDKR, Faiman, PVWatts) and the estimators are scikit-learn. It is that *every number the interface shows can be traced back to an input, every default is on the table for the user to override, and nothing modelled is ever presented as measured*. Those three constraints produced the specific architectural choices documented above — the assumption ledger, the ban on target-lag features, the panel-count-wins rule, the empirically determined interval offset, the conformal calibration of prediction intervals, the comparison endpoint that refuses to rank incomparable runs.

Practically, the platform runs on a laptop with no API keys, produces an answer in a few seconds, and ships with 270+ automated tests, an ASGI-ready backend and a Next.js frontend that can be deployed behind any reverse proxy. Its future potential lies in adding real-time monitoring, ground-truth validation, additional languages, and the sequence-model class explicitly declared as future work in the model registry.

33 Glossary

Term	Meaning
PV	Photovoltaic — direct conversion of sunlight to electricity.
kW / kWh	Kilowatt (power, instantaneous) / kilowatt-hour (energy, integral).
kWp	Kilowatt-peak — DC rating of a panel or system at standard test conditions.
DC / AC	Direct current (from panels) / alternating current (grid). The inverter converts DC to AC.
DC/AC ratio	Ratio of installed DC capacity to inverter AC capacity. Ratios > 1 trade a small amount of clipping for better winter/morning performance.
GHI	Global Horizontal Irradiance — total solar flux on a horizontal surface (W/m^2).
DNI	Direct Normal Irradiance — beam flux on a surface normal to the sun.
DHI	Diffuse Horizontal Irradiance — sky-scattered flux on a horizontal surface.
POA	Plane-of-Array irradiance — total flux reaching the tilted panel plane.
AOI	Angle of Incidence — angle between the sunbeam and the panel normal.
P_{max}	Panel maximum-power-point rating in watts.
$V_{\text{oc}}, I_{\text{sc}}$	Open-circuit voltage, short-circuit current at STC.
$V_{\text{mp}}, I_{\text{mp}}$	Voltage / current at the maximum-power point at STC.
PR	Performance Ratio — actual AC energy divided by ($\text{POA} \cdot \text{rated capacity}$), a dimensionless efficiency of the whole system.
Capacity factor	Annual energy divided by ($\text{rated capacity} \cdot 8760$), the average utilisation.
Specific yield	Annual kWh per kWp; a location-and-orientation-specific figure.
Clear-sky index (k_t)	Ratio of observed to clear-sky irradiance; used as the ML target.
Bifacial	Panel producing from both faces; rear-side gain depends on albedo.
TOPCon / HJT	Advanced silicon cell architectures with improved efficiency and temperature coefficient.
Degradation	Annual reduction in output as panels age (default 0.5 %/yr).
LCOE	Levelised Cost of Energy — lifetime cost divided by lifetime generation (currency per kWh).
PICP / PINAW / CRPS	Prediction Interval Coverage Probability, Prediction Interval Normalised Average Width, Continuous Ranked Probability Score.

Term	Meaning
Conformal prediction	Distribution-free interval calibration; gives finite-sample marginal-coverage guarantees.
Reanalysis (ERA5)	Retrospective global weather record produced by assimilating observations into a physical model on a grid.

34 References

Papers cited by the platform

1. [P1] Hobbs, W. & Joshi, S., "*An operational hybrid framework for hour-ahead solar irradiance forecasting*" (referenced throughout as the source of the clear-sky-index target, Erbs decomposition and Faiman cell temperature).
2. [P2] Mabodi, T. & Hammujuddy, Y., "*Machine-learning models for monthly global horizontal irradiance*" (source of the KNN and RF hyperparameter defaults).
3. [P3] Lyu, C. & Eftekharijad, S., "*Copula-based feature selection for solar-power forecasting*" (source of the 87° zenith day-mask threshold and the weather-regime conditioning).
4. [P4] Rosales Huamani, J. et al., "*Stacked-ensemble regression for solar irradiance*" (source of the stacking-ensemble combination).
5. [P5] Vijay Babu, R. et al., "*Feature importance for solar-power forecasting*" (source of the Gradient Boosting hyperparameters and the AOI permutation-importance finding).
6. [P6] Hayajneh, A. et al., "*LSTM-based short-term irradiance forecasting on ESP32*" (listed as future work — sequence models and TinyML edge deployment).

Standards, methods and data sources

- Dobos, A. P. (2014). *PVWatts Version 5 Manual*. NREL/TP-6A20-62641.
- Faiman, D. (2008). *Assessing the outdoor operating temperature of photovoltaic modules*. Progress in Photovoltaics.
- Erbs, D. G., Klein, S. A. & Duffie, J. A. (1982). *Estimation of the diffuse radiation fraction for hourly, daily and monthly-average global radiation*. Solar Energy.
- Haurwitz, B. (1945). *Insolation in relation to cloudiness and cloud density*. J. Meteorol.
- Michalsky, J. J. (1988). *The astronomical almanac's algorithm for approximate solar position*. Solar Energy. (NOAA Solar Calculator formulation.)
- Kopp, G. & Lean, J. L. (2011). *A new, lower value of total solar irradiance: Evidence and climate significance*. Geophysical Research Letters. (Solar constant 1361 W/m².)
- Hersbach, H. et al. (2020). *The ERA5 global reanalysis*. Q. J. R. Meteorol. Soc.

Services and libraries

- Open-Meteo Historical Weather API — open-meteo.com (CC-BY 4.0).
- Open-Meteo Geocoding API — GeoNames (CC-BY 4.0).
- OpenStreetMap Nominatim — ODbL / usage policy.
- FastAPI — fastapi.tiangolo.com.
- Next.js — nextjs.org.
- scikit-learn — Pedregosa et al., JMLR 2011.
- Pandas — McKinney, W., PyData 2010.
- NumPy — Harris, C. R. et al., Nature 2020.

35 Appendix — API and Configuration

Consumer estimate endpoints

Method	Path	Purpose
GET	/api/meta/interview	The interview as data (personas + questions).
GET	/api/meta/place_search?q=	Place-name search.
GET	/api/locations/reverse?latitude=&longitude=	Reverse-geocode a coordinate.
POST	/api/estimate	Create an estimate (optionally save).
GET	/api/estimate/{id}	Read a saved estimate.
POST	/api/estimate/{id}/update	Edit assumptions and re-run.
POST	/api/estimate/{id}/rename	Rename a saved estimate.
DELETE	/api/estimate/{id}	Delete a saved estimate.
GET	/api/estimates?limit=	Recent estimates on this server.
GET	/api/estimate/{id}/report	Plain-text professional report.
POST	/api/estimate/compare	Run multiple scenarios in one call.

Analysis-console endpoints (research surface)

Method	Path	Purpose
POST	/api/analysis	Create an analysis (fetch, quality, features, train, evaluate).
GET	/api/analysis/{id}	Summary of the analysis.
GET	/api/analysis/{id}/quality	Data-quality report (eight corruption detectors).
GET	/api/analysis/{id}/predictions	Actual-vs-predicted series for the test set.
GET	/api/analysis/{id}/performance	Metrics, folds, baselines, error decompositions, intervals.
GET	/api/analysis/{id}/explain	Permutation importance, partial dependence.
GET	/api/analysis/{id}/anomalies	Physical / residual / ramp / drift detectors.
POST	/api/analysis/{id}/forecast	Short-horizon forward forecast.
POST	/api/analysis/{id}/scenarios	Counterfactual analysis with pathway decomposition.
GET	/api/analysis/{id}/report	Markdown research report.
GET	/api/experiments	List persisted experiments.
POST	/api/experiments/compare	Compare comparable experiments; refuses incomparable ones.

Sample estimate request (abridged)

```
{
  "user_type": "farm",
  "mode": "quick",
  "goal": "install",
  "location": { "query": "Warangal, India" },
  "consumption_method": "equipment",
  "equipment": [
    { "key": "led_light", "count": 20, "hours_per_day": 5 },
    { "key": "ceiling_fan", "count": 8, "hours_per_day": 8 }
  ],
  "pumps": [
    { "horsepower": 5, "head_metres": 22, "hours_per_day": 6 }
  ],
  "area": { "value": 800, "unit": "sqm" },
  "system": { "panel_key": "mono_perc" },
  "farm": { "pump_horsepower": 5, "pump_head_metres": 22 },
  "save": true,
  "label": "Warangal 5HP pump"
}
```

Sample response shape (abridged)

```
{
  "estimate_id": "e_9x7...",
  "saved": true,
  "location": { "label": "Warangal, Telangana, India", "latitude": 17.98, "longitude": 79.6,
"timezone": "Asia/Kolkata" },
  "currency": { "code": "INR", "symbol": "₹" },
  "system": {
    "capacity_kwp": 6.60,
    "panels": { "count": 12, "watts": 550, "model": null },
    "orientation": { "tilt_deg": 22.3, "azimuth_deg": 180, "method": "searched" },
    "inverter": { "ac_capacity_kw": 5.5, "verdict": "...", "severity": "info" },
    "sizing": { "binding_constraint": "demand", "reason": "..." }
  },
  "generation": {
    "annual_kwh": 10230,
    "monthly_kwh": [ ...12 values... ],
    "monthly_ranges": [ ...12 (low,high) pairs from that month's own variability... ],
    "daily_profile_kwh": [ ...24 values... ],
    "specific_yield_kwh_per_kwp": 1550,
    "capacity_factor": 0.177,
    "clipped_fraction": 0.006
  },
  "balance": { "self_consumption_pct": 63, "solar_offset_pct": 78, "annual_import_kwh": ...,
"annual_export_kwh": ... },
  "economics":{ "total_capex": 396000, "annual_savings_year1": 62000, "payback_years": 6.3,
"roi_pct": 214, "lcoe_per_kwh": 3.1 },
  "emissions":{ "co2_avoided_tonnes_per_year": 7.4, "equivalences": { "cars_off_road": 1.6,
"trees_planted": 340 } },
  "uncertainty":{ "expected": 10230, "lower": 9200, "upper": 11300, "confidence": "medium",
"factors": [ ... ] },
  "farm": { "daily_pump_hours_typical": 5.8, "daily_water_m3": 218, "irrigable_ha": 1.4
},
  "assumptions": [ ...ledger rows, editable=True where applicable... ],
  "warnings": [],
  "data_sources": [ ...provenance metadata... ]
}
```

Configuration reference

See `backend/app/config.py` for the full dataclass structure. All settings are environment-variable driven with safe defaults; nothing here is secret.